

CSCE799

Sprint 1 – Thesis Review

Dynamic Compilation of Synchronous Real Time Hashes & The Cocky Cipher Suite



SPRINT PLAN

- SPRINT0(08/12) – Baseline 542 Cocky Investments App Overview.
- SPRINT1(09/15) – Discussion Of Semester And Progress Update
- SPRINT2(10/15) – Secure Application Delivery Complete & LateDraft
- SPRINT3(12/01) – Final Write Up And Assurance



CSCE799 (6CH)

- We need to prepare one or more academic papers sufficient for publication with a focus on Application Security of Mission Critical Applications.
- We need to be able to test and verify our work.
- Our work needs to justify the credit hours associated with the project, has to have a robustness component.
- We have to take into account critical commentary on the basis of our work.



AGENDA

- Synch up on Sufficiency For This term.
- Initial Plan
- Modified Plan
- Status of App, Book, Paper.
- Demo of Gen4 App.

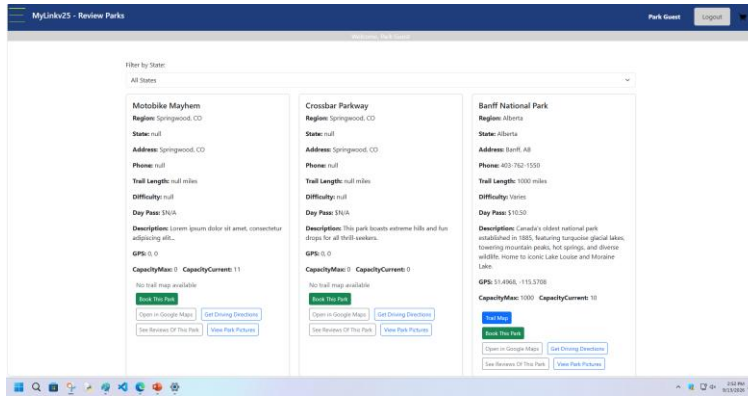


INITIAL PLAN

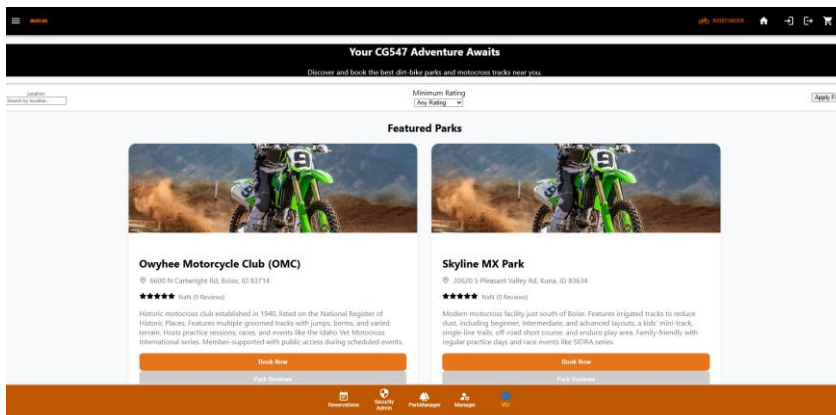
Summer 2026



547G (FALL 2026) – CG PARKS APP



HTML/JS App as Website
ReactJS App As Class Requirement



CG React UI with Nested JSON Objects

Azure
Cloud
API
Layer
(SQLite3 DB)

C# API Controller
North/South Interfaces



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

CG UI REQUIRED COMPLEX DTOS (EMBEDDED JSONS)

- CG App posted Southbound(Towards REST GW) -> Transactions which included the User Record, The Product, The Transaction, Credit Card Info – In a Nested JSON.
 - Requirements For Backoffice to Accept this Data Structure on HTTP Post, and Demux it to several tables. (Very Complex Service Arch).
 - This JSON is Not Encrypted Today But Runs Over HTTPS to Azure Service Architecture on Windows.
 - App writes Transaction data to Local Storage which is unencrypted before the post.
- CG APP makes a Northbound Request For Data On Initialization For User Data, Product Data, Park Data, Customer Reviews. CG Provided the JSON format we had to emulate for the App to work.
 - The Backoffice has to make multiple select statements over EF
 - Send back muxed data in semistructured JSON format.
- This was a VERY COMPLEX project where I was asked to compete against teams of five by myself.... But I had a working SSO controller from 590.



INITIAL PLAN

- Take Apps Developed Over 3 Semesters with CG in 547U/590G/547G and move them to secure software.
 - These are all REST Centric Client-Server Architectures.
 - Focus on Encrypting Northbound/Southbound Data Structures defined by CG for scalable IoT solutions
 - The Initial Designs created cookies, session fragments, and data in databases which was not encrypted.
 - HTTPS for Client server in 547U/547G is optional, and is provisioned by Microsoft in all services built in Azure by default.
 - HTTPS for Client-Server in 242 on Render is NodeJS is provisioned with HTTPS and OpenSSL base by default.
 - Publish with CG the Exact Code Required to Encrypt DTOs.

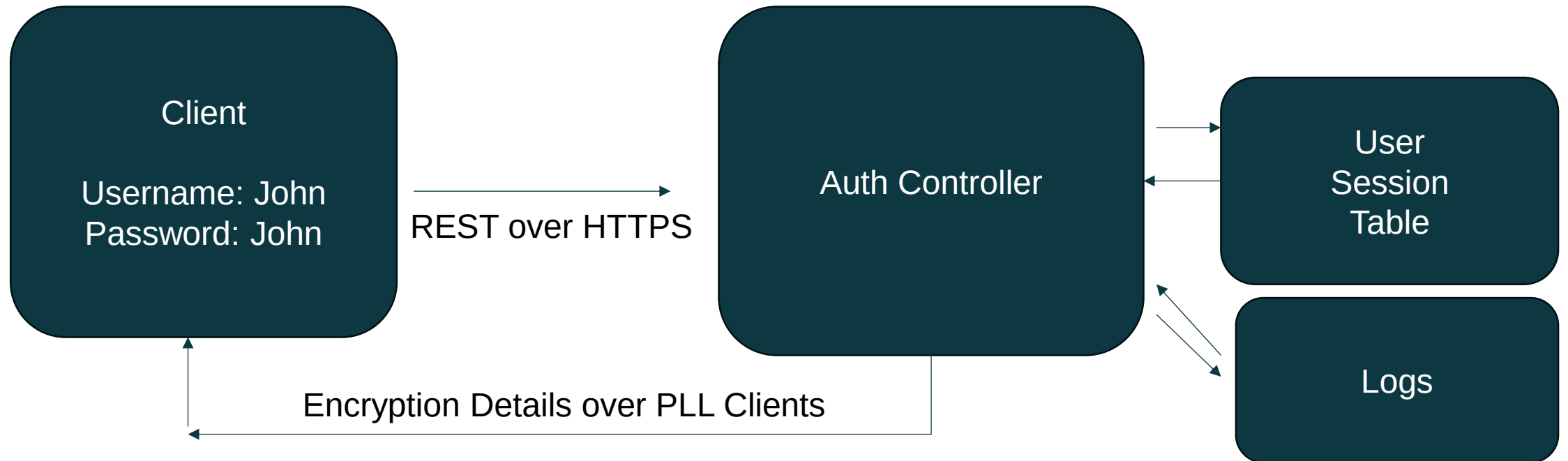


JOINT PROPOSAL WITH CG CONSULTING ON SECURITY DESIGN FOR LARGE SCALE INTERNET OPS

- Initially we planned to take the semi-secure 547GBikes Application we made with CG consulting in class which had a working Azure SSO system, and configure REST payload encryption between clients and servers. (Security Upgrade).
- DTO Architectures
 - Using a Northbound(Client Setup) nested JSON format which includes app configuration and encrypt the Northbound payload to every client.(1)
 - Using a Southbound(Client Transactions) nested JSON format which includes payment transactions and encrypt the Southbound payload to ever server.(2)
- OTHER Security Requirements Using PLL
 - Using the same cipher, encrypt initial login processes(username/password)(3) -> SB#2, and returns NB#2 which included a PLL cached user record, session key, and hash parameters also encrypted(4).
 - Using the same cipher encrypt cookies(5), local browser storage(6), and anything in memory which was stored temporarily(7).
- Since the network was already encrypted with HTTPS and commercial ciphers, I believed that PGP Application standards simply needed to obsecure all data from the human eye to meet PCI standards/HIPAA standards, and Fisma standards.
- Since I had just taken a class on Ciphers(CSCE715), I thought rather than do a Caesarian cipher a substitution cipher with a SHIFT INTO THE EXTENDED ASCII range would make it unintelligible to humans and would make this possible in the simplest possible manner.



INITIAL PLAN... THE AUTH CONTROLLER



Uid: 1
Fullname: John Stritzinger
Email: stritzj@email.sc.edu
Opt 1PSK: 96
Opt 2 Dynamic: $\text{cubeRt}(\text{Random}(96:150))+53$



RESULT....

- Initial Plan... CG Consulting and USC publish a globally famous paper on how to scale Internet clients, and secure them...using nested JSON objects and DTOs(Data Transfer Objects).
- Why? CG has 400K global consultants, and anything they write as an internal document would be delivered all over the world, and internally cited.
- CG(Columbia) who teaches CSCE547/CSCE590 and their engineers said yes...and were helping...but Departmental calls resulted in regional VP's at CG to say no... they didn't want to publish global security standards to the research community.



BUILDING SECURE SOFTWARE.....

- Becomes a Security Project Management Effort.
 - Phase I – Build An Application over REST.
 - Phase II – Add SSO/Social Logins & PLL
 - Phase III – Add Session Based Network Tools for Security
 - Phase IV – Make Efforts to Secure Networks, Disks, Temp Files.
 - Phase V – Build Tools To Audit Data Flows
 - Phase VI – Document Compliance to Federal Standards



THE PROJECT BUILDS ARCHITECTURAL REQUIREMENTS....

- Encrypt Networks
- Encrypt Temporary Files, Caches, and Browser Storage
- Encrypt Memory Based Data Structures
- Encrypt Data Stored in Databases
- Serial JSONs in Complex Form NorthBound
- Serialized JSONs in Complex Form SouthBound
- Demux JSONs Northbound
- Demix JSONs Southbound
- Build a General Purpose PGP Cipher for All Tasks which is developer controlled.



CIPHER PROPOSAL FOR SECURE APPS

Initial Proposal



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

A CAESARIAN CIPHER HAS VERY SMALL BRUTE FORCE ISSUES

- In an English Alphabet, and an equal substitution pattern of 1 to 26 there are only 26 brute force options....
- Brute Force Breaking
 - Apply an Offset to a message... compare it to an English dictionary.
 - There are 26 different offsets.
 - The total matching words with the greatest count = The offset.
 - There are only 26 permutations.
 - It's a three pass algorithm (pass 1 apply an offset, pass 2 send each word to a dictionary program, pick the result, send back the translation for the result. A human can decompose a message in less than an hour manually... and with a computer perhaps 10 seconds.



CARNEGIE MELON PROPOSED IMPROVEMENTS

- Mtext -> Ctext by letter... but in an array.. With no fixed spacing.
- So 26! Possible permutations in the array.
 - Eg A->Z ,B->C
 - CM proposes this is a “Caesarian Family cipher”... there is a translation from one letter to another but its not the same spacing.
 - Casear was 3 initially and moved to 7 at some point in time.
 - But what if Caesar had a different offset to each legion?(30 Legions)....
 - So then there would be 30X26 Keys still not good enough eh?

Example:

```
1 Plain : A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
2 Cipher: Q M Z T H P X R K N C L V B O J D I Y A F W G E U S
```

As arrays:

```
1 plain = ['A','B','C','D','E','F','G','H','I','J','K','L','M','N','O','P','Q','R','S','T','U','V','W','X','Y','Z']
2
3
4 cipher = ['Q','M','Z','T','H','P','X','R','K','N','C','L','V','B','O','J','D','I','Y','A','F','W','G','E','U','S']
5
```

Mappings:



WE HAVE PROPOSED A COCKY CIPHER-B TOOLSET

- Unique 1Key Algorithm which use ASCII Extended Codes.
- Designed for PGP (Pretty Good Privacy) Not Commercial Tools
- Work in Conjunction with HTTPS Network Transport
- Unbreakable Across the Wire, PGP For DevOps, Change Control
- Defeat Word Analysis
- Defeat AI Tools
- Have 256! Permutations instead of 26! Permutations in the Carnegie Mellon version.



COCKY CIPHER

A simple substitution plan to ASCII extended is done in near real time in any modern programming language is still better than Caesarian algorithms as they defeat dictionary attacks and its PCI compliant... no English text anywhere...

• Before Encryption

- Letter "A" -> ASCII Value(65)

• After Encryption

- Extended Code + Shift 96
- $65 + 96 = \text{ASCII} \rightarrow 161$
- Inverted Question Mark



• Before Encryption

- Park the Car in the Harvard Yard

• Ex2 -> After Encryption

°ÁÒÈ€ÒÈÀ€ÆÁÒÈ€Í€ÒÈÀ€°·ÁÒÈ
ÁÒÈÁ€°ÁÒÈ

- So this is very different from a normal Caesarian cipher is the translation to ASCII is an encoding in itself.
- There are 26 letters in the English Alphabet but these have 52 ASCII values. And if we can use literally any ASCII value to substitute... the Permutation of the mapping isn't 26! As Carnegie Mellon discusses... its actually 255!/52! Just by shifting to ascii...

Complete ASCII 0–255 Character Chart

Here is the full ASCII table (0–255) with decimal, hex, octal, binary, symbol, and name for every code, including control characters, printable letters/digits, and extended ASCII [\[x\] ASCII table +2](#).

How to Use

- 0–31: Control codes (e.g., NUL, SOH, BEL)
- 32–127: Standard printable ASCII (letters, digits, punctuation)
- 128–255: Extended ASCII (Windows-1252/Latin-1 supplement)

Full Table (Dec | Hex | Oct | Bin | Char | Name)

ASCII printable characters (character code 32–127)

Codes 32–127 are common for all the different variations of the ASCII table, they are called printable characters, represent letters, digits, punctuation marks, and a few miscellaneous symbols. You will find almost every character on your keyboard. Character 127 represents the command DEL.

DEC	OCT	HEX	BIN	Symbol	HTML Number	HTML Name	Description
32	040	20	00100000	SP	 	&space;	Space
33	041	21	00100001	!	!	!	Exclamation mark
34	042	22	00100010	"	"	"	Double quotes (or speech marks)
35	043	23	00100011	#	#	#	Number sign
36	044	24	00100100	\$	$	$	Dollar
37	045	25	00100101	%	%	&percent;	Per cent sign
38	046	26	00100110	&	&	&	Ampersand
39	047	27	00100111	'	'	'	Single quote
40	050	28	00101000	(	(	&lparen;	Open parenthesis (or open bracket)
41	051	29	00101001	)	)	&rparen;	Close parenthesis (or close bracket)
42	052	2A	00101010	*	*	*	Asterisk
43	053	2B	00101011	+	+	+	Plus
44	054	2C	00101100	,	,	,	Comma
45	055	2D	00101101	-	-	&hyphen-minus;	Hyphen-minus
46	056	2E	00101110	.	.	.	Period, dot or full stop
47	057	2F	00101111	/	/	&slash;	Slash or divide
48	060	30	00110000	0	0	&zero;	Zero
49	061	31	00110001	1	1	&one;	One
50	062	32	00110010	2	2	&two;	Two
51	063	33	00110011	3	3	&three;	Three
52	064	34	00110100	4	4	&four;	Four
53	065	35	00110101	5	5	&five;	Five
54	066	36	00110110	6	6	&six;	Six
55	067	37	00110111	7	7	&seven;	Seven
56	070	38	00111000	8	8	&eight;	Eight
57	071	39	00111001	9	9	&nine;	Nine
58	072	3A	00111010	:	:	:	Colon
59	073	3B	00111011	;	;	&semicolon;	Semicolon
60	074	3C	00111100	<	<	<	Less than (or open angled bracket)
61	075	3D	00111101	=	=	=	Equals
62	076	3E	00111110	>	>	>	Greater than (or close angled bracket)
63	077	3F	00111111	?	?	?	Question mark
64	100	40	01000000	@	@	&at;	At sign
65	101	41	01000001	A	A	Á	Uppercase A
66	102	42	01000010	B	B	&Bacute;	Uppercase B
67	103	43	01000011	C	C	Ć	Uppercase C
68	104	44	01000100	D	D	&Dacute;	Uppercase D

DEC	OCT	HEX	BIN	Symbol	HTML Number	HTML Name	Description
128	080	80	10000000	À	€	À	Uppercase A with grave accent
129	081	81	10000001	Á		Á	Uppercase A with acute accent
130	082	82	10000010	Â	‚	Âumflex;	Uppercase A with circumflex accent
131	083	83	10000011	Ã	ƒ	Ã	Uppercase A with tilde accent
132	084	84	10000100	Ä	„	Ä	Uppercase A with umlaut
133	085	85	10000101	Å	…	Å	Uppercase A with ring
134	086	86	10000110	Æ	†	Æ	Uppercase AE ligature
135	087	87	10000111	Ç	‡	Ç	Uppercase C with cedilla
136	088	88	10001000	È	ˆ	È	Uppercase E with grave accent
137	089	89	10001001	É	‰	É	Uppercase E with acute accent
138	090	8A	10001010	Ê	Š	Êumflex;	Uppercase E with circumflex accent
139	091	8B	10001011	Ë	‹	&Etilde;	Uppercase E with tilde accent
140	092	8C	10001100	Ü	Œ	Ü	Uppercase U with umlaut
141	093	8D	10001101	Ý		Ý	Uppercase Y with acute accent
142	094	8E	10001110	Þ	Ž	&Thorn;	Uppercase Thorn
143	095	8F	10001111	ß		&Scharf;	Uppercase Sharp S
144	096	90	10010000	à		á	Lowercase a with grave accent
145	097	91	10010001	á	‘	á	Lowercase a with acute accent
146	098	92	10010010	â	’	âumflex;	Lowercase a with circumflex accent
147	099	93	10010011	ã	“	ã	Lowercase a with tilde accent
148	100	94	10010100	ä	”	ä	Lowercase a with umlaut
149	101	95	10010101	å	•	å	Lowercase a with ring
150	102	96	10010110	æ	–	æ	Lowercase ae ligature
151	103	97	10010111	ç	—	ç	Lowercase c with cedilla
152	104	98	10011000	è	˜	è	Lowercase e with grave accent
153	105	99	10011001	é	™	é	Lowercase e with acute accent
154	106	9A	10011010	ê	š	êumflex;	Lowercase e with circumflex accent
155	107	9B	10011011	ë	›	&etilde;	Lowercase e with tilde accent
156	108	9C	10011100	ü	œ	ü	Lowercase u with umlaut
157	109	9D	10011101	ý		ý	Lowercase y with acute accent
158	110	9E	10011110	þ	ž	þ	Lowercase thorn
159	111	9F	10011111	ÿ	Ÿ	&ygrave;	Lowercase y with grave accent
160	112	A0	10100000		 	 	Non-breaking space
161	113	A1	10100001	°	¡	°	Degree symbol
162	114	A2	10100010	¹	¢	¹	Superscript one
163	115	A3	10100011	²	£	²	Superscript two
164	116	A4	10100100	³	¤	³	Superscript three
165	117	A5	10100101	´	¥	&apost;	Apostrophe
166	118	A6	10100110	µ	¦	µ	Micro sign
167	119	A7	10100111	¶	§	¶	Paragraph symbol
168	120	A8	10101000	·	¨	•	Bullet
169	121	A9	10101001	¸	©	¸	Lowercase c with cedilla
170	122	AA	10101010	¹	ª	¹	Superscript one
171	123	AB	10101011	²	«	²	Superscript two
172	124	AC	10101100	³	¬	³	Superscript three
173	125	AD	10101101	´	­	&apost;	Apostrophe
174	126	AE	10101110	µ	®	µ	Micro sign
175	127	AF	10101111	¶	¯	¶	Paragraph symbol



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

LOOKING CLOSER HOW MUCH BETTER ASCII CODE IS OVER TRADITIONAL CAESARIAN CIPHERS

- Park the Car in the Harvard Yard. With Straight ASCII

Using traditional 7-bit ASCII (case-sensitive, preserving uppercase and lowercase exactly as written), the string:

```
1 Park the Car in the Harvard Yard
2
```

encodes to the following decimal ASCII values:

```
1 P = 80
2 a = 97
3 r = 114
4 k = 107
5   = 32
6 t = 116
7 h = 104
8 e = 101
9   = 32
10 C = 67
11 a = 97
12 r = 114
13   = 32
14 i = 105
15 n = 110
16   = 32
17 t = 116
18 h = 104
19 e = 101
20   = 32
21 H = 72
22 a = 97
23 r = 114
24 v = 118
25 a = 97
26 r = 114
27 d = 100
28   = 32
29 Y = 89
30 a = 97
31 r = 114
32 d = 100
33 . = 46
```

As a single ASCII sequence:

- With Extended 96 Shift

```
1 Park the Car in the Harvard Yard
```

Character-by-character examples:

```
1 P = 80 -> 176
2 a = 97 -> 193
3 r = 114 -> 210
4 k = 107 -> 203
5 (space)=32 ->128
```

Applying a shift of +96 to every character gives the extended ASCII codes:

```
1 176 193 210 203 128 212 200 197 128 163 193 210 128
2 201 206 128 212 200 197 128 168 193 210 214 193 210
3 196 128 185 193 210 196
```

In hexadecimal:

```
1 B0 C1 D2 C8 80 D4 C8 C5 80 A3 C1 D2 80
2 C9 CE 80 D4 C8 C5 80 A8 C1 D2 D6 C1 D2
3 C4 80 B9 C1 D2 C4
```

As extended ASCII characters (Code Page dependent):

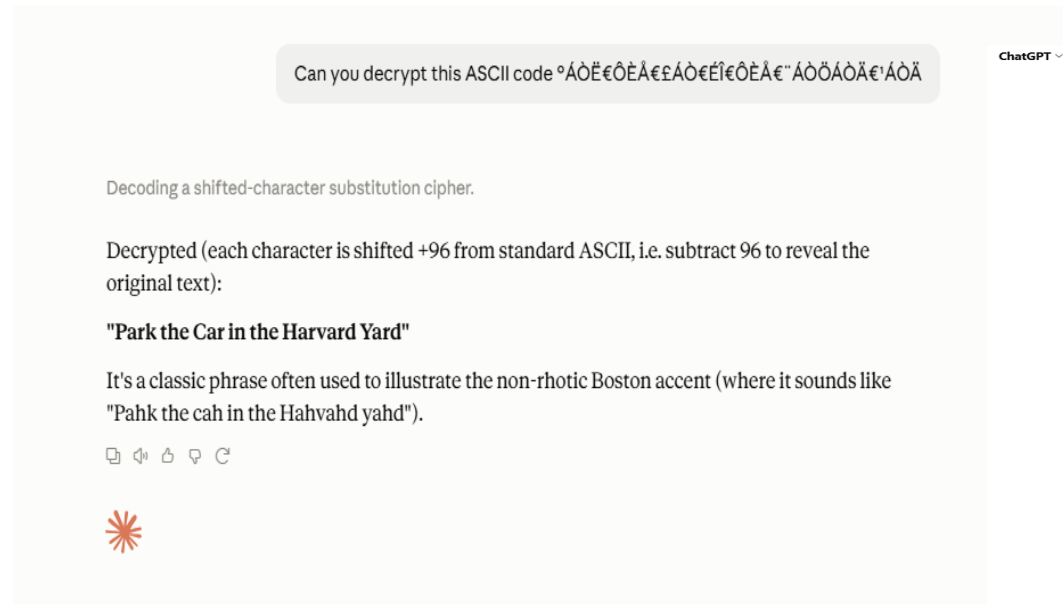
```
1 *ÃÐËÖÈÃÊËÃÖÊËËÖÊË*ÃÐÃÖÊ*ÃÐÃ
```

(Note that characters above 127 display differently depending on the extended ASCII code page being used, so the numeric values are usually the safest representation.)

Microsoft Copilot Using ChatGPT5 Models responds in under 5seconds
But only if you tell it the format of the algorithm as a clue. ASCII
Substitution Cipher.



ASKING AT TOOLS TO DECRYPT TO SIMPLE SHIFT...



< 10 seconds with prompts

Without prompts ChatGPT... ????

Any Human looking at that Gibberish of control codes... will say... that's cipher text... why do I want to do that?



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

THE ORIGINAL PLAN IS COMPLETE...

- The auth controller works....
- It can handle simple Caesarian ciphers where there is a single 1:1 mapping between letters and numbers.
- It can handle complex ASCII ciphers with a shift which delivers 255!/52! Choices of permutations.
- It can handle DotNet Crypto libraries which have most of the OpenSSL tools and a few unique ones.



CHALLENGES IN COMPLEX SHIFTS....

- Client and server have to know what the shifts are....
- Client and server have to either have them presaved or “PSK” or passed over the network....
- The difference is more significant over HTTP than HTTPS.
- Shift of 96 has to be sent across the wire or in the source code.
- What if we want the shift to be 97...or 98 or some other number.
- What if we want to have a different shift for each stream?



WE HAVE PROPOSED 4 ADV SUB CIPHERS

- USE ASCII CODES
- USE 5 KEYS INSTEAD OF 1 BASED ON LETTER POSITIONS
- CAN WORK IN PSK, OR NETWORK AUTH ORIGINATED.
- ADVANCED COCKY VARIANTS
 - ADV1 – Use NTP protocol to generate 5 keys.
 - ADV2 – Use NTP And Session Manager to generate new keys for every session. Meaning 1M clients get 1M permutations.
 - ADV3 – Use Session Manager to Send Complex Primes For Dynamic Calculation.
 - ADV4 – Send Equations As Strings and Dynamically compile 5 keys.



ADVISOR RESPONSE

09/14/2026



INPUT FROM ADVISOR....

- Why go through the use of another cipher type?
- What are you gaining from not using a commercial tool to do the same thing?
- Can you propose a paper on a new cipher based on a Caesarian model without getting laughed at?



LETS LOOK AT THE CIPHER OPTIONS

Class	Variant	QuantumProtection	AI Confusion	Specifics	Keys	ClientTypes	Server Types	NetworkEncryptor	REST Support	Comments	Permutations	ManInMiddle	Combinations with HTTPS
Substitution Cipher	Caesarian	No	No	Fixed Shift	1	PSK, Network Key	PSK, Network Key	Possible	YES Network Keys At Auth/Bootstrap	PSKs don't come across wire	26 in Basic Form	Protected Via HTTPS	Yes JSON Serialization Before
BlockSubstitutionCipher	Vignere			Variable Key Size	1	PSK, Network Key	PSK, Network Key	NotLikely	YES	PSKs don't come across wire	KeyLengthDetermines Combinations	Protected Via HTTPS	Yes JSON Serialization Before
MonoAlphabetic	Adv Caesar	No	Partial	LetterShuffleOverArray	1	PSK, Network Key	PSK, Network Key	Possible	YES Network Keys At Auth/Bootstrap	PSKs don't come across wire	26! Permutations	Protected Via HTTPS	Yes JSON Serialization Before
CockyCIPHERBase(Proposed)	Pro Caesar	RequiredToBreak	Yes	LetterShuffleIntoASCII	1	PSK, Network Key	PSK, Network Key	Yes	YES Network Keys At Auth/Bootstrap	PSKs don't come across wire	255! Permutations	Protected Via HTTPS	Yes JSON Serialization Before
CockyCIPHERBasell(Proposed-PGP-S)	Pro Caesar	No	Yes-High	Complex WordShift, LetterShift	5	NTP Driven	NTP Driven	Yes	Yes Network Time	PSKs don't come across wire	255*5, WordBreak, Dict Break	Protected Via HTTPS	Yes JSON Serialization Before
CockyCIPHERBasell(Proposed-PGP-A)	Pro Caesar	No	Yes-High	Complex WordShift, LetterShift	5	NTP Driven	NTP Driven	Yes	Yes Network Time	PSKs don't come across wire	255*5, WordBreak, Dict Break	Protected Via HTTPS	Yes JSON Serialization Before
CockyCIPHERAdv(Proposed)	Pro Caesar	RequiredToBreak	Yes	MultLetterShuffleIntoASCII	5	PSK, Network Key	PSK, Network Key	Yes	Yes Network Time	PSKs don't come across wire	255!exp5	Protected Via HTTPS	Yes JSON Serialization Before
CockyDynamic	Pro Caesar	Yes	Yes-High	MultipleEquationsLogin	5	Dynamic Net	Dynamic Net	Yes	Yes Network Time	KeysNeedToBeCached	InfiniteUnbreakable	Protected Via HTTPS	Yes JSON Serialization Before
Commerical Technologies													
RSA-2048	DotNetCrypto, OpenSSL	Yes	No	Commercial Libraries	1	Network Key	Network Key	Yes	YES				
RSA-3072	DotNetCrypto, OpenSSL	Yes	No	Commercial Libraries	1	Network Key	Network Key	Yes	YES				
DES	DotNetCrypto, OpenSSL	No	No	Commercial Libraries	1	Network Key	Network Key	Yes	YES				
3DES	DotNetCrypto, OpenSSL	No	No	Commercial Libraries	1	Network Key	Network Key	Yes	YES				
AES-256	DotNetCrypto, OpenSSL	Yes without MITM	No	Commercial Libraries	1	Network Key	Network Key	Yes	YES				
AES-512	DotNetCrypto, OpenSSL	Yes without MITM, EP	No	Commercial Libraries	1	Network Key	Network Key	Yes	YES				
Elipitcal Curves	DotNetCrypto, OpenSSL	No	No	Commercial Libraries	1	Network Key	Network Key	Yes	YES				



RESPONSE....

- If we add commercial tools as well as our own proposed mods... then we are in an operational mode instead of a research mode...
- And this makes making secure software a Security Project rather than something of intellectual value.
- It's a trivial task if you own the REST GW server and have admin rights... but in the real world its not a given.
- Dynamic programming isn't PGP its trying to build a commercial cipher to compete with AES, 3DES, etc. But it has an infinite set of options effectively.

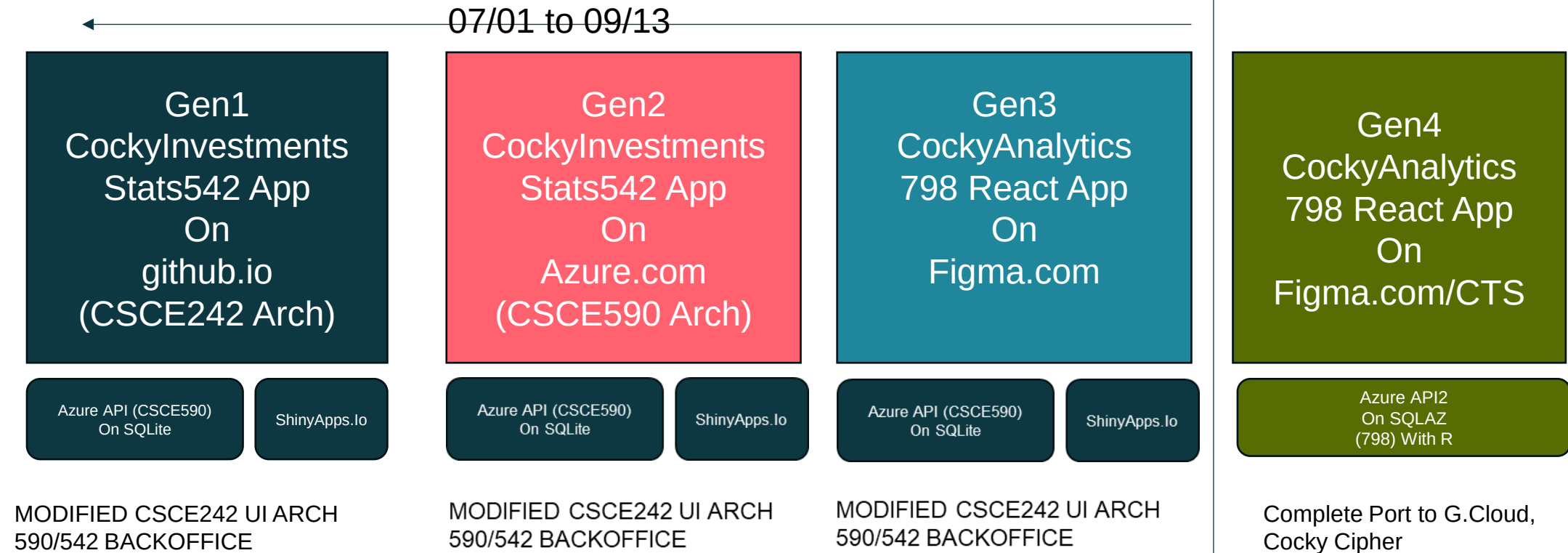


MODIFIED PLAN

09/13/2026



GENERATIONAL ARCHITECTURE IMPROVEMENTS

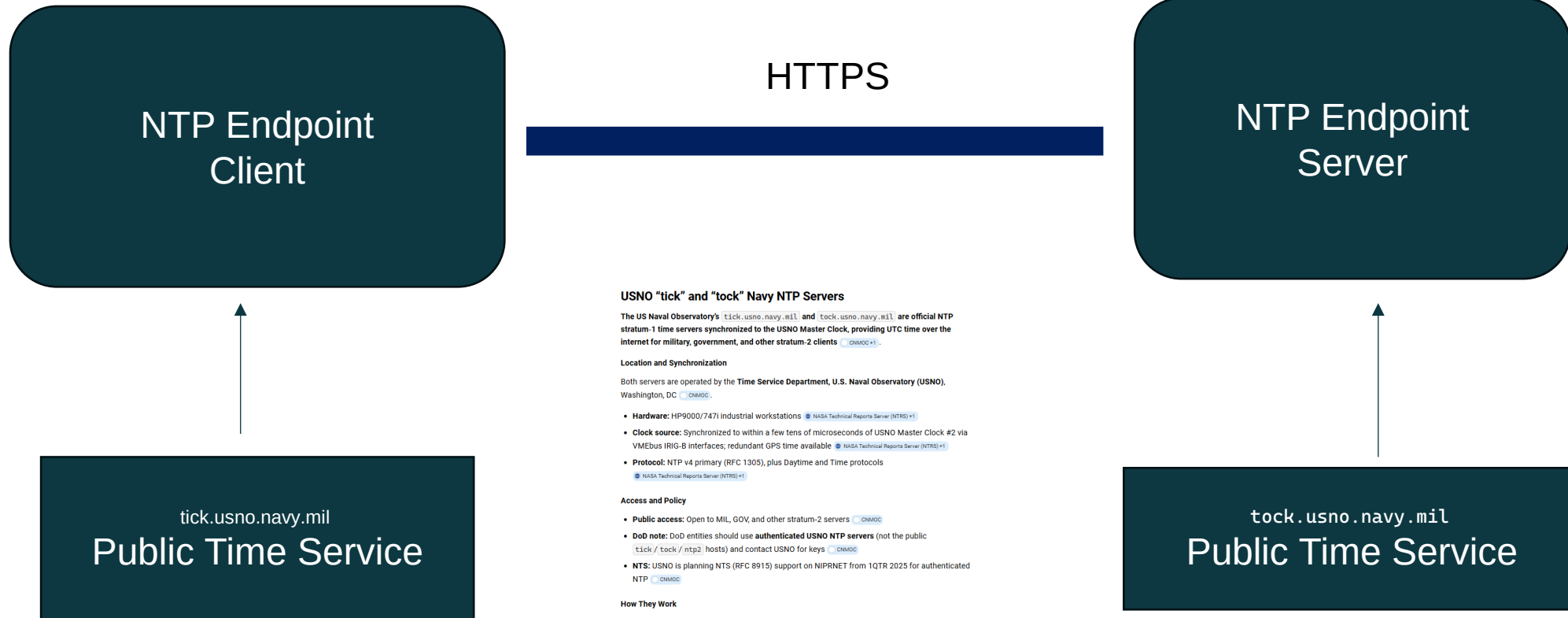


EX Equation Form -> Day X Month Mod 128 – [Mutator] = 21 for C1= Shift

A KEYLESS DYNAMICALLY COMPILED CIPHER?

09/13/2026

09/13/2026



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

UPGRADING THE COCKY CIPHER... SO ITS NOT...CAESARIAN ...OR IS DYNAMIC

- Moving from 1 Key to 5 Keys based on NTP Protocol.
- Supporting Synchronous, and Asynchronous modes by offsetting base time.
- Moves the Algorithm to $255!/52! \times 255!/52! \times 255!/52! \times 255!/52! \times 255!/52!$ permutations and is unbreakable by AI engines even if you tell them it's a substitution cipher.
- If you change the cipher by stream for a million clients you could have $255!/52! \times 5 \times 1m$.
 - Example Key 1 Shift is $96 + \text{Day of Month}$.
 - Example Key 2 Shift is $96 + \text{Month}$
 - Example Key 3 Shift is $96 + \text{cubert}(\text{Month} \times \text{Day})$
 - Example Key 4 Shift is $96 + \text{cubert}(\text{Year}) + \text{squareroot}(\text{Month} \times \text{day})$
 - Example Key 5 Shift is $96 - 2 \times \text{Day}$ all would result in 5 unique keys
- TEXT[0], TEXT[1], TEXT[2], TEXT[3], TEXT[4], TEXT[5] having a different key.



THE RESULT....

- 1) The Same Sentence... sent over the wire... will change every single day...
 - 2) Stored Cached Users will automatically be useless on Day2 (Auto Expiration) even without browser clearing.
 - 3) Forensic analysis of PCs would need massive processors to try to decrypt data... based on older encryption keys/ciphers which are almost unbreakable....
-
- A) On their own Shifted single key ASCII Advanced Substitution are unintelligible to both humans and AI engines alike.
 - B) Dynamically compiled codes which could have a different algorithm PER CLIENT are near unbreakable and having more streams is not helpful.
 - C) We think that PGP Privacy is achieved with a single key cipher into ASCII extended ranges which is PCI and HIPAA compliant. It passes cipher text and not plain text to Network Ciphers which are already near immune from hacking. We think for real time processing we have met our burden of proof. It's a significant achievement.
 - D) Standard fixed keys are very much more valuable for data storage.



STREAM RANDOMIZATION

- Caesar writes one message to all 30 legions....
 - His Chief Messenger encrypts it into 30 streams 1 per legion.
 - Regional Managers send the encrypted form to each legions.
 - Legions Respond back with their own confirmation.
 - The Chief Messenger receives 30 messages of confirmation all encrypted differently.
 - The Chief Messenger decrypts all of them, and summarizes them for Caesar.
 - We can do this by the Auth controller responding with a cipher or an offset from one of many in the database.



CARNEGIE MELONS ALT ALGORITHM .. LETS LOOK.....

- Carnegie Mellon says there are approximately 26! Permutations of a standard English alphabet in a 1:1 mapping..... But that's only if one letter maps to 1 letter.... The number is still humungous..... IE there is no fixed substitution.
- To achieve the Carnegie Mellon cipher I have to send a 26 position array with characters across the wire.
 - cipher char[26] -> C++ dim statement.

On the
Order of
 10^{26}
 $403 =$
septillions

This value is often entered in cryptography because there are exactly $26! \approx 4.03 \times 10^{24}$ possible monoalphabetic substitution ciphers using the 26-letter English alphabet.

👍 👎 🗑️ ...

can you add commas

1

26!

$\approx 403,291,461,126,605,635,584,000,000$

🔗

🗑️

↔️

⌵

Plain Text

Scientific notation:

1

26!

$\approx 4.032914611266056 \times 10^{26}$

2

🔗

🗑️

↔️

⌵

Plain Text

With digit grouping:

1

$403,291,461,126,605,635,584,000,000$

🔗

🗑️

↔️

⌵

Plain Text

That's about **403 septillion** in the short scale naming system.

👍 👎 🗑️ ...

But an ASCII
Substitution is....

We are picking 52 values(cap sensitive)
for 255 choices...or 26 values for 255
choices...cap insensitive.

Its $255!/52!$ Permutations.... For brute force....

[illegible]

**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

THE COCKY CIPHER....

- AN ASCII EXTENDED CIPHER CAN BE USED FOR ANY COMMUNICATIONS OR CLIENT PURPOSES IN PGP FORM.
- IT CAN BE USED AS A NETWORK ENCRYPTOR AND ELIMINATE > 90% OF ALL KNOWN THREATS AND MAKES WIRESHARK ALMOST USELESS.
- CAN BE DRAMATICALLY INCREASED IN COMPLEXITY AFTER LOGIN WITH DYNAMIC COMPILATION TO UNBREAKABLE? ON A PER SESSION BASIS.
- IT'S A TWO PASS ALGORITHM...
 - PASS ONE: LETTERS TO ASCII.
 - PASS TWO: LETTERS TO ENCRYPTION ARRAY MAPPING.
- BUT LETTERS CAN BE MAPPED TO ANY ASCII VALUE UP TO 255.
 - AND WELL THIS COULD USE NOT LATIN ALPHABETS TOO... WHICH IS EVEN MORE COMPLICATED...



THEORETICALLY....

- IBMS largest quantum computer...

how about IBMs largest quantum computer

IBM's largest publicly announced quantum processor was IBM Quantum Condor, with 1,121 physical qubits, unveiled in late 2023. It was the first IBM chip to exceed 1,000 qubits. [kissasoft +1](#)

To put that in perspective:

$$2^{1121} \approx 3.8 \times 10^{337}$$

So a 1,121-qubit quantum state can mathematically occupy a superposition of roughly:

1 3.8×10^{337} states

at once.

However, it is important not to interpret that as:

1 3.8×10^{337} calculations per second

because current quantum computers are noisy, and the number of useful computations is limited by error rates and circuit depth. Researchers instead focus on metrics such as reliable quantum operations, gate fidelity, and logical qubits.

For comparison:

System	Physical Qubits	Possible Quantum States
255-qubit	255	$2^{255} \approx 5.8 \times 10^{76}$
1,121-qubit Condor	1,121	$2^{1121} \approx 3.8 \times 10^{337}$

The number 3.8×10^{337} is vastly larger than estimates for the number of atoms in the observable universe (about 10^{80}). The power of a quantum computer comes from manipulating amplitudes across that enormous state space, not from brute-forcing every possibility independently.

>> Single Key
ASCII Code
Has More
Permutations than
the worlds largest
quantum computer.

And it would still require a 2nd dictionary pass run or squared... So on a large message unfeasible.

what is $255!/52!$

$$\frac{255!}{52!}$$

Coding and executing >

equals:

[illegible]

Scientific notation:

$$1 \approx 4.1543844474925365 \times 10^{454}$$

So $\frac{255!}{52!}$ is a 455-digit number.



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

FROM AN ALGORITHM TO DYNAMIC CODING

- Dynamic Hashes As Parameters
 - We have to take strings entered on an application and encrypt them in real time.
 - If the functions are compiled then you cant change the keys unless you pass them as parameters.
 - Encrypt(text, key) where key in this example is “shift96”.
- Dynamic Compilation of a Different Function....
 - IE I login... and the response from the auth controller isn't an integer... it's a different algorithm.
 - The shift is the square root of 96, or the cube root of 96 or 3x the cube root of 96...



THESIS -> DYNAMIC COMPILATION OF ADVANCED ASCII SUBSTITUTION CIPHER?

If An AI Engine cant decrypt an ASCII code and its used to dealing with ASCII codes and computer translations.....

Then what will humans think.... Can we get to PCI compliance very quickly?

We think yes.... We asked two CSCE102 students to look at the short string... 1 one TA... and all were stumped... they didn't know where to start....and this means they don't know how to ask an AI engine either.

Javascript Real Time
Compilation of a
Function using Mathjs

```
JavaScript

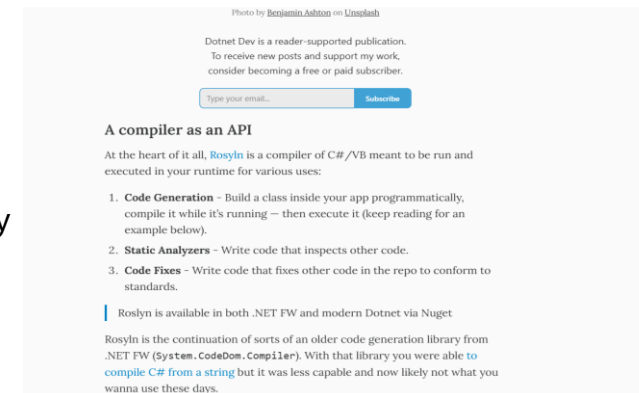
import { evaluate, parse } from 'mathjs';

// Server sends just the expression
const serverExpression = "x^2 + 3";

// Client compiles it safely
const compiled = parse(serverExpression).compile();

// Evaluate with different values
console.log(compiled.evaluate({ x: 2 })); // 7
console.log(compiled.evaluate({ x: 5 })); // 28
```

REST
Gateway
Roslyn



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA

OUR PLAN FOR THIS COURSE

- Take a class Stats542 Analytics App and try to make it industrial grade. (A Graduate Task).
 - Improve Scalability
 - Dramatically Improve Security
 - Add Customer Feature Requirements (Customer Service, Profiles, etc)
- Upgrade the Application to support self driving Customer Analytics (IE Stock Screening to Profile Design)
- Upgrade the Application to prep for ML Trading Operations or their equivalents at other banks.
- Make the Application FIPS/FISMA HIGH compliant and support PCI/HIPAA standards which would be required to pass security audits at a trading house.
 - This requires that both network, and application encryption would be required.
 - This requires a full analysis of client risks and security requirements (banded devices)



CONCLUSION

- A Secure Application in the Financial Services world is a 798 project here on campus on its own.
- An Operations Project is probably not a thesis unless it becomes a book for publication and for ongoing training.
- A Research paper needs to be something unique, and we think per stream dynamic encryption is certainly unique.
 - The CockyCipher we developed using network time we think is near unbreakable without a state sponsor of security tools.
 - We have up to 13 papers which would come out of our security proposal.
- Dr. Valefar asked me not to present anything related to 547G as initially intended.



THANKS!

John Stritzinger
Graduate Researcher
stritzj@email.sc.edu
843-352-3459



**Molinaroli College of
Engineering and Computing**
UNIVERSITY OF SOUTH CAROLINA